SOLPS-ITER v. 3.0.10 and 3.1.2 Release Notes

April 2nd, 2026

These Release Notes give a summary of the changes from SOLPS-ITER versions 3.0.9/3.1.1 to 3.0.10/3.1.2 and give information about how to transition an older SOLPS-ITER run to be continued with the 3.0.10 or 3.1.2 code versions. Users are encouraged to read this document carefully and thoroughly before proceeding with use of SOLPS-ITER 3.0.10 or 3.1.2. Users should also consult the updated version of the SOLPS-ITER Manual that accompanies this distribution and other documents posted to the SOLPS-ITER web site and Confluence page. In particular, the minutes from the SOLPS-ITER User Forums will provide more detailed information about the features discussed below. For full details of all updates, please consult the SOLPS-ITER Manual and GIT repository logs. In addition to the items listed below, the new code version also contains lots of bug fixes, new documentation, updates to the SOLPS-ITER Manual and configuration files, code clean-up, and output clarification/prettifying improvements, as reported/requested by users and developers since the last release of the `3.0.9e` and `3.1.1e` versions. Some of the changes that have occurred on the **Eirene** side are documented in the updated **Eirene** manual.

Note: The 3.0.10 and 3.1.2 code versions differ mostly in the numerical scheme used for solving the plasma transport equations. The 3.0.10 version relies on a 5-point stencil, while the 3.1.2 version allows for use of a 9-point stencil (with backward compatibility to the 5-point stencil solution method). They are otherwise nearly identical in terms of features and physics model. Notably, they couple to the same **Eirene** version. For the remainder of this document, unless explicitly stated, the 3.0.10 moniker applies to both the 3.0.10 and 3.1.2 code versions. These release notes do not apply to the Wide Grids 3.2.1 SOLPS-ITER code version.

If you are transitioning from SOLPS-ITER v.3.0.6 or earlier to v.3.0.10, then you should also consult the v.3.0.7, v.3.0.8, and v.3.0.9 Release Notes and their addenda (see the SOLPS-ITER Confluence page for a full list).

The code is freely distributed as open-source software on https://github.com/iterorganization/SOLPS-ITER. Some optional code components may not (yet) be available as open-source, such as the ADAS data files, however these are accessible to civilian state and academic institutions within the ITER Member States, provided they have signed an IMAS Access Request with the ITER Organization for that purpose. You can consult the list of signatory institutions here. Other entities may also sign an IMAS Access Request, subject to approval by the ITER Legal Office. Signatory institutions have the option to sponsor third party users. The code is licensed under EUPL. You will find the SOLPS-ITER `LICENSE` document in the top directory of the code distribution, and as an Appendix to the SOLPS-ITER Manual. There is an additional `LICENSE` document in each of the ADAS, **B2.5**, **Carre**, and **DivGeo** submodules which covers their usage as stand-alone programs. Users requiring access to the **Eirene** repository as part of the SOLPS-ITER package also need to abide by the **Eirene** license (CC-BY-NC-ND for plain users, and the one found in the `EPL` file in the top **Eirene** directory for developers), which is managed by Forchungszentrum Jülich in Germany (check www.eirene.de for more details).

Users that have not done so already are encouraged to subscribe to the SOLPS-ITER Slack channel (solps.slack.com). Simply send an email request to the IO Responsible Officer (xavier.bonnin@iter.org) to receive an invitation to join if you need one.

1. How to obtain SOLPS-ITER v.3.0.10:

Because the code is now distributed from GitHub and not the git.iter.org repository anymore, simply create a new tree and clone the code from https://github.com/iterorganization/SOLPS-ITER. By default, this will yield the *master* code version. If you wish to point instead to the *develop* branch, then use the *solps-iter_update_develop* script, or *solps-iter_update_3.1.2_release* for the 3.1.2 code version. To return to the *master* version, use the *switch_to_master* script.

For subsequent updates, users should use *solps-iter_update, solpsiter_update_develop*, or *solps-iter_update_3.1.2_release*, depending on which branch they are using, and only once per update.

IMPORTANT: When switching code versions, as this may involve a number of configuration file changes, new environment modules, and several added/renamed/moved/removed source code files, it is likely that the compilations launched by the update scripts will fail the first time around. If this is the case, start afresh in a new window, load the updated SOLPS-ITER environment as usual by means of the *setup.csh* file, and try again to compile. If it fails once more, then do a `gmake clean` first and compile again. Recall that you can use `gmake clean_XXX` to remove all objects associated with any `XXX` build, for example `gmake clean_solps_mpi`.

2. New features:

This new release contains many small bug fixes, sometimes mentioned in the User Forum debriefings, which are not listed below. Some code clean-up and alignment corrections were also made to keep the distance between the various main code branches as small as possible and simplify comparisons across them. Here we mention mostly the new features that provide code capability changes from version 3.0.9.

2.1. GitHub distribution:

- As mentioned above, this is the first version of SOLPS-ITER meant for open-source distribution. When cloning the SOLPS-ITER GitHub repository, you can use either the HTTPS or SSH methods. The latter is recommended, for which you must provide you public SSH key to the GitHub server. Using the HTTPS method will only allow you to clone a read-only copy of the code, and you will not be able to push any of your own changes back to the repository. The *.gitmodules* file has been modified so that the cloning method used for the SOLPS-ITER repository will be inherited by the cloning instructions for the submodules, if using the usual `git submodule init` and `git submodule update` commands. All new code development should now occur on GitHub. Users are encourages to use the `Issues` tab to report bugs or other problems encountered while running the code, and the `Discussions` tab to trigger debate about new features they would like to see added and how to implement them.
- The case examples have been moved to a Zenodo record that can be consulted at https://zenodo.org/communities/solps/ and the instructions and *Makefile* for downloading them modified accordingly.

2.2. Support for IMAS Data Dictionary 4.1.x and SimDB run archival/retrieval:

- The code is compatible with IMAS Data Dictionary versions up to `4.1.1`, GGD version `1.14.0`, and Fortran Access Layer `5.5.0`. It is also recommended to update to IDStools version `2.4.0` and IMAS-ParaView version `2.3.0`. When prompted to produce IMAS output, the code will write out, in addition to the previously supported IDSs, the *plasma_profiles*, *plasma_sources*, and *plasma_transport* IDS, which will contain essentially the same information as *edge_profiles*, *edge_sources*, and *edge_transport*, respectively (and are eventually meant to replace them). The *data_description* IDS is obsolete and will not be created, instead the metadata it contained can now be found in the *summary* IDS.
- When an IMAS data entry is part of the run's output, the code will create a SimDB manifest file (*manifest.yaml*). This file is intended to contain a list of all the input and output files involved in the run, as well as a unique alias identifier and verbose description field. Users should check that the information the code wrote in the manifest is sufficient and complete. Then, the file can be pushed to a SimDB database by means of the new *ingest_simulation_to_SimDB* script. This feature requires IMAS Access Layer 5.

2.3. Coupling to a single Eirene version for all B2.5 distributions:

- All the main SOLPS-ITER code distributions now use coupling to the same **Eirene** version, derived from the Milestone Version (MsV) and available as open-source. The main difference is that the Wide Grids distribution will use the interface routines from the *couple_SOLPS_WG* directory, while the structured code versions rely on the interface routines in *couple_SOLPS-ITER*. A couple of other variables (`L_MACRO` and `IFC TRI`) have also been added to the main code, but are only needed by the Wide Grids coupling scheme. The *Makefile* can normally detect which coupling interface to use, but can be told which one by means of the `INTDIR` environment variable, meant to contain the absolute path of the interface directory to be used, if not using `CMake` to compile **Eirene**.

2.4. Extension of the capabilities for simulating time-dependent ELM events:

This extension is based on the already existing possibility to define ELM transport coefficients profiles in *b2.transport.inputfile* (through the `tdata (3,:,:,:)` fields). However, so far only periodic step changes of the transport coefficients were possible. Now:

- More sophisticated dynamics are possible, including exponential/logarithmic ramp-up/down phases, with plateau phases in between.
- All time scales defining the periodic alteration of the transport coefficients (start of ramp-up, end of ramp-up, start of ramp-down, end of ramp-down) can be different for the 9 types of transport coefficients defined via *b2.transport.inputfile*.
- The periodic changes in the transport can be poloidally localized around the outer midplane through imposing a ballooning-type dependency (also possibly different for each coefficient set).
- All the above mentioned options can be enabled via parameters in *b2.transport.inputfile*. All changes are fully backwards-compatible, so the old ELM model can still be used as before.

2.5. Electric potential equation solution for time-dependent runs:

- For cases where the electric potential at the targets is expected to vary very fast, such as for ELM pulses reaching a target, a new solution method using internal iterations is available to converge the potential equation on the time-step, with the maximum number of iterations allowed set by the `b2news_m_iter_pot` switch. Convergence criteria can be set with the `b2urb_epsr` (for relative accuracy) and `b2urbp_epsa` (for absolute accuracy) switches (default value is `0.01` for both).

2.6. Added flexibility (and improved output) when invoking the neoclassical transport model via `b2tqna_user_transport=8`:

- It is possible to add the `NEOART`-calculated transport contributions to the radial transport coefficients (particle diffusivity/convective velocity, electron/ion thermal diffusivities, etc.), or a to a subset of them, via the switches `b2tqna_neoclassical_xxx` (with `xxx` = `dna`,`vla`,`hci`,`hvi`,`hce`,`hve`). The poloidal range, in the confined region, over which this applies can be defined by via the switches `b2tqna_neoclassical_iystart/end`, and the list of affected species through the logical array `NEO_NS_SET` of size (0:NS-1) within the new namelist *b2.neoclassical_transport.parameters*.
- It is possible to perform the calculation of the neoclassical transport coefficients only every `N` **B2.5** time steps, through the new switch `b2tqna_neoclassical_time_steps` (default = `1`). When they are not re-calculated, the previously calculated transport coefficients are used. Note that, for now, the switch works as intended only when both `b2mndt_nstg0` and `b2mndt_nstg1` are `1`, while it works also for multiple 2nd level iterations.
- It is possible to print the neoclassical transport coefficients calculated by `NEOART` (all of them, not only those really added to the **B2.5** radial transport coefficients) to a new *b2neo.nc* output file after each of its calls.

2.7. Expanded blobby transport model with `b2tqna_user_transport=6` option:

- New model parameters `b2tqna_*_cnv` can be used to mimic blobby transport radial convection velocity. See the code documentation for details.

2.8. New Eirene averaging scheme:

- A new averaging scheme, activated by the `eirene_averaging_multiple_calls` switch, by which **Eirene** can be called multiple times within a single **B2.5** iteration. Although this, of course, would mean a slower code execution, it is an effective means to reduce sampling bias when other methods are not available, like when doing time-dependent runs in which one wishes as accurate an impurity particle balance as possible. It is found that this method provides better results than simply increasing the number of **Eirene** trajectories followed, for subtle reasons due to the numerical treatment of the time census stratum.

2.9. New analysis plotting scripts:

- The *2d*, *2da*, *2dt*, and *2dt_av* printing/plotting scripts have been expanded to support also the new multi-species fields now contained in *b2time.nc*.
- A new *2dds* script has been added, to plot profiles from *b2time.nc*, against the (automatically selected) proper radial coordinate.

- The *2d_plots* and *2d_plots_av* scripts have been expanded to include now also species-resolved time traces (plotted together according to which isonuclear sequence the species belong). For consistency and clarity, the generated PostScript files have been renamed from *b2time2d(_av).ps* to *b2time2d_time_traces(_av).ps*.
- The *2d_profiles* script has been expanded so that it prints *.last10* (pro)files also for multi-species fields (namely one file for each fluid species). Since, for simulations with a large number of species, the number of outputted files could become very large, potentially "polluting" a lot the run directory, these files are now put in a *2d_profiles/* subfolder of the run directory. Additionally now, when the script is invoked, all profiles are also plotted to a new PostScript file, named *b2time2d_profiles.ps*, in the same fashion as already done for *b2time2d_time_traces.ps*.
- A new *2dt_tracing* script has been added, which plots time traces of quantities contained in the *.trc* tracing files (for now only those contained in *tracing/blnn_SPb.trc* and *tracing/blne.trc*). Additionally, all time traces are automatically plotted to the PostScript files *particle_balance.ps* and *energy_balance.ps* via the auxiliary new scripts *particle_balance_tracing* and *energy_balance_tracing*.
- For consistency, new scripts for automatically plotting the norm of the residuals of the ion and electron energy equations (*reshe* and *reshi*, respectively) have been added, in the same fashion as the already existing scripts *resco* and *resmo*. Additionally, all of the 4 scripts are automatically called through an auxiliary new script *plot_residuals*, which plots the norm of all residuals to a new PostScript file *residuals.ps*.
- A new auxiliary *groups_species* script has been added, which prints the ordeded number of species contained in each isonuclear sequence (as something like that was needed for some of the new above mentioned plotting scripts). E.g. executing the script in the run directory of a `D+N+He+Ne` case outputs `2 8 3 11`.
- A new *summarize_run_tracing* script is available that calls the *2d_profiles*, *2d_plots*, *particle_balance_tracing*, *energy_balance_tracing*, *plot_residuals*, and *last10* scripts.

2.10. Installation and setup scripts:

- Configuration files have been updated to provide environment variables needed for the `CMake` compilation of **Eirene**, like the location of various support libraries.
- New configuration files have been provided for the IPP `gfortran` environment, the EFGW machine (still labelled with `ITM` as its hostname) and the DLUT cluster with `ifort64` toolchain.
- The **IMAS-ParaView** visualization tool and the Catalyst debugging engine now also work on the ITER SDCC cluster with the `ifort64` toolchain, and in the EFGW `gfortran` environment.
- The default toolchain for the `IPP` and `EFGW` hosts has been changed to `gfortran`.
- The `LEUVEN` cluster files have been updated to reflect the new architecture there.
- A new generic `LINUX` hostname can be used for installations on laptops running Linux, relying on the `gfortran` toolchain.
- The *install_dependencies* script has been improved and should be able to install locally all needed (freely available) external libraries.
- The recommended `MSCL` library version to use is `1.2.5`.

2.11. Job and batch compilation submission scripts:

- A new batch compilation script *efgwmake* is available for use on the `ITM` EUROfusion Gateway machine.
- The *itermake* and *efgwmake* scripts will now launch the compilation from the `$SOLPSTOP` directory, no matter from which location where they were invoked.
- The *pitagorasubmit* script will allow for job submission on the `PITAGORA` machine.
- The *ippsubmit* and *mpcdfsubmit* scripts have been expanded to allow for launching multiple runs within a single SLURM job.

3. Changes and extensions to the physics model:

Please be mindful that the changes to the default physics model described in this section will most likely cause some evolution of existing 3.0.9 solutions. The magnitude of that change will depend on the specifics of the cases at hand. The benchmarks and sample cases available as part of the code distribution have all been recomputed using the 3.0.10 physics model.

The fundamental constants have been updated to the CODATA 2022 standard.

After careful consideration, it was found that the **B2.5** coordinate system has a COCOS index of 4, as opposed to 6 as previously thought.

A new "DT" hydrogenic species with mass 2.5 is now supported for **B2.5** runs.

3.1. Non-marginal sheath boundary conditions:

- Drift-compatible non-marginal sheath boundary conditions have been implemented (`BCCON=14`, `BCMOM=13`, `BCENI=15`), which are triggered if `MOMPAR(,,2)>0.5` (in the same way as for the traditional type `3` sheath BCs). **IMPORTANT:** For the electron heat BC (`BCENE=15`), please note that the previously unused `ENEPAR(,1)` parameter is now meant to modify the electron heat sheath transmission coefficient. Since this parameter used to have a value for the old style BC `BCENE=3`, it may not be zero depending on the history of your *b2.boundary.parameters* file, so you should check your settings. Please see the SOLPS-ITER Manual for a full description.

3.2. Extension of the Grad-Zhdanov closure scheme for heavy species:

- An extension to the Zhdanov-Grad closure scheme has been implemented to include neoclassical transport corrections for heavy impurity species, compatible with the standard drifts treatment implemented in **B2.5**. These can be activated by means of the `b2tral_Zhdanov_nc` switch. When that switch is set to `1`, the corrections will be applied for ions from isonuclear sequence number `b2tral_Zhdanov_nc_spec` and charge greater than or equal to `b2tral_Zhdanov_nc_min` (default 6). These corrections are recommended for heavy impurities, Neon (Z=10) and above, and with a charge state of at least +6.

4. Changes to the input files:

4.1. Changes to the Eirene input files:

- A new switch `NLSPCSPT` (20th logical in the second line of block 1 of the **Eirene** fixed format input file) allows for tallying sputtering sources according to the projectile species.
- For **Eirene** runs with a time census stratum, the size of the stratum is now always defined to be `NPRNLI`, if `NPTST=0`, to avoid problems arising from runaway sputtering cascades. If `NRPNL` or `NTIME` are set to zero, then the time census stratum is turned off.
- The **Eirene** input files produced by the **Uinp** case build-up program will now contain the `NLSPCSPT` switch as well as the `NEVNTMAX` and `NVNMXTR` numbers which limit very long trajectories after a given number of events.
- The `NFULLRUN` switch is deprecated and its spot in the **Eirene** fixed format input file is used by `NEVNTMAX` instead.

4.2. Upper limit for ADAS rates:

- In addition to the limits imposed to the extrapolation of ADAS rates by the `adas_extrap` switch, a hard limit is imposed on the cooling and recombination rates of e^{-20} eV.m^3/s.

5. Changes to the output:

5.1. Additional Eirene output:

- One of the electron particle balance contributions tallies, namely `SNE_PIEL`, the electron source due to interactions between electrons and molecular ions, was missing in previous versions. It is now included in the *balance.nc* file and in the IMAS output. Users need to recall, however, that, in the **Eirene** model that is used by SOLPS-ITER, molecular ions are not followed and are assumed to immediately dissociate at their birth location, so any associated sources are approximate.
- The **Eirene** sputtering tallies can now be split according to the species that caused the sputtering, using the `NLSPCSPT` switch.
- The **Eirene** log output has been made to look essentially the same whether the fixed form ASCII or the JSON format of the input file is used.

5.2. Modifications to b2plot:

- The **b2plot** variable `ue` is now computed in an identical fashion as in the main code.
- The format of the numbers printed out in the wall loading tables produced by **b2plot** (with the `wlld` command) and in the *tracing/* output files was changed so 10^{-100} will be written as `1.000E-100` instead of `1.000-100`, which could confuse scripts reading these numbers.

5.3. New time-traced quantities:

- Various **B2.5** and **Eirene** fluxes and anomalous transport coefficients fields on the **B2.5** grid were added to *b2movies.nc* and *eirenemovies.nc* files, respectively.
- Several quantities were added to *b2time.nc*, in particular with regards to multi-species quantities and **Eirene**-related quantities, such as densities for all fluid species (`nasep[a|i|m]`), atomic and molecular densities from **Eirene** (`d[a|m]bsep[a|i|m]`), atomic and molecular temperatures from **Eirene** (`t[a|m]bsep[a|i|m]`), density profiles for all fluid species (`na3d[a|i|l|r]`), atomic and molecular density profiles from **Eirene** (`d[a|m]b3d[a|i|l|r]`), and atomic and molecular temperature profiles from **Eirene** (`t[a|m]b3d[a|i|l|r]`). The midplane transport coefficients profiles (`dn3d[a|i]`, `vx3d[a|i]`, `vy3d[a|i]`, `vs3d[a|i]`) and target particle flux profiles (`fn3d[a|i]`) are now given for all species, not only the main ion. This addition, because it requires adding new dimension definitions in the header of the *b2time.nc* file, is NOT backward-compatible and thus requires the user to restart the time trace archival when continuing a 3.0.9 run with the 3.0.10 code version (by removing/renaming the old *b2time.nc* file or setting `b2mndr_stim` to a zero or positive value).
- To help in the self-containedness of *b2time.nc*, it now includes the radial profile coordinates (`dsi`, `dsa`, `dsr`, `dsl`), target areas (`dsR`, `dsL`) and poloidal contact areas (`dsRP`, `dsLP`) at the midplanes and/or targets.
- To avoid issues on case-insensitive file systems where files like *dsr* and *dsR* would be identical, the latter are renamed to *dsRT*. Same for *dsL* becoming *dsLT*.

6. Getting a case restarted with v.3.0.10:

As a consequence of all the changes above, in order to take full advantage of the new 3.0.10 features, users wishing to continue their older runs should keep in mind the issues listed below. If running an existing 3.0.9 case, continuation of this case with the 3.0.10 version should be possible without any special additional preparations (except checking the `ENEPAR` setting for BCs with `BCENE=15`), although it is recommended to correct the input files following the information given above or run a **b2yt** conversion step to apply all automated corrections. Because of the various code changes and bug fixes implemented in this new version, users should expect some slight changes to their solutions. Of course, to facilitate converting older cases or building new cases from scratch, the **Uinp**, **b2yt**, and **b2sxdr** programs have been modified to produce input files already compatible with version 3.0.10, which should not preclude a visual inspection of all relevant input files, as is best practice.

1. If you are restarting a case from input files created using code from before November 2019, you should delete the existing *fort.1[0-5]* files as there was an internal change in **Eirene** that has modified the content of these files.
2. If you were previously using NetCDF3, you may not be able to append to your older **.nc* files and would then need to delete/rename them.
3. To take advantage of the new NetCDF quantities that can now be saved in *b2time.nc* and associated files, the older files should not be carried over (otherwise only the previously defined quantities will be saved), or the time history should be reset by giving the `b2mndr_stim` switch in *b2mn.dat* a positive or zero value.

7. Additional notes for developers:

7.1. Pull requests:

- When making a Pull Request on GitHub, please make sure that the branch from which you are making the Pull Request is listed on the ITER Organization GitHub, which will then trigger the Continuous Integration (CI) testing at ci.iter.org. Not doing so will delay review of your Pull Request, since it cannot be merged formally unless and until the CI tests are passed.

7.2. Parallelization improvements:

- The mapping of the **Eirene** sources to **B2.5** arrays is done inside OpenMP loops in *b2mod_eirene.F* to speed up the code and the loops have also been rewritten to facilitate vectorization.

7.3. Miscellaneous:

- A new auxiliary *replace_links* script has been added, which recursively replaces all symbolic links in a specified directory (and its subdirectories) with the real file or folder to which they are linked, keeping the same name as the original link. This can be quite useful when one wants to transfer a run and/or *baserun* directory to a new cluster/computer/environment.
- The new `write_manifest` routine in *b2mod_ual.F90* contains a list of the potential I/O files that may be included in a SimDB manifest. Thus, when adding a file to the code workflow, this list needs to be updated accordingly.
- The *ioflush* routine in **Eirene** no longer relies on its C implementation and has been moved to the *user-routines/user_*/ioflush_usr.F* file where any local implementation requirements can be applied.
- The **Eirene** routines can have either a lowercase or an uppercase extension (*.f[90]* or *.F[90]*). Uppercase extensions are used to indicate to the *Makefile* that pre-processing of the file is required. In cases where multiple copies of the same routine exist (e.g. *interfaces* or *user* routines), then, if one of them requires pre-processing, it is recommended that all files with the same name be given an uppercase extension.
- Two separate **nc2text** executables can now be built, with and without debugging options.